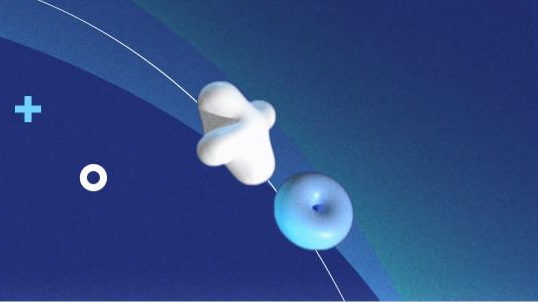


ASSOCIATE ARCHITECT (조별과제)

지역 상품권 결제 플랫폼

첨단기술아카데미

CONTENTS



00 과제개요

01 요구공학: ex3 mini-QAW

02 OOAD: ex1~9 (iteration1~3)

03 Architecture설계: ex4~5(View), ex6~7(Tactic), ex8(Design)

04 Design Patterns: Pattern 5개 적용

05 Architecture평가: ex1~2 (ATAM)

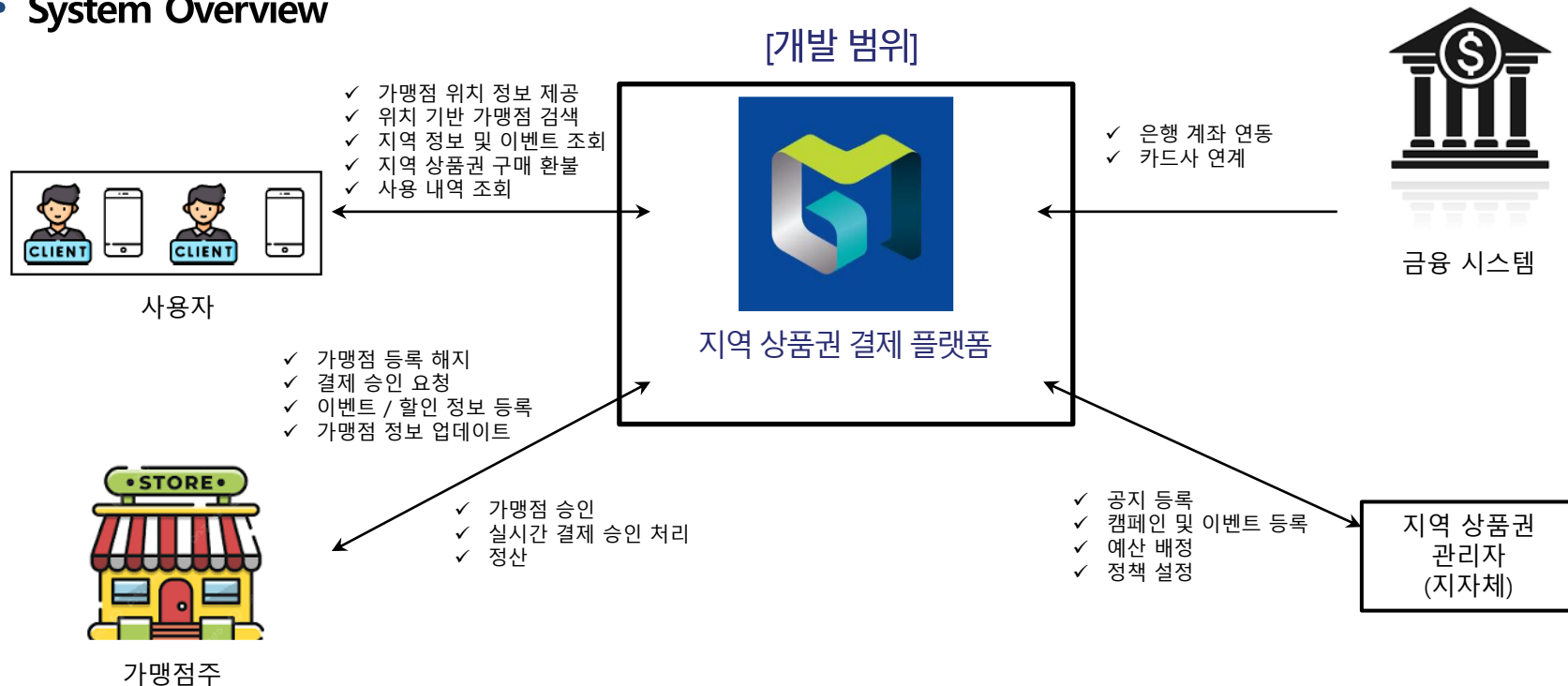
• Project Overview

- 지역사랑상품권 발행 규모는 코로나19 이후 3조원에서 6조원으로 2배 증가하며 급속한 성장세 보임
- 전국 243 개 지자체 중 231 개 지자체에서 지역상품권을 운영하며 전국적 확산 추세
- 모바일 결제 방식 선호도 증가로 디지털 지역상품권 이용 확대
- 단순 결제 기능에서 지역 정보 제공 및 맞춤형 추천 서비스로의 플랫폼 진화 필요
- 제한된 예산으로 효율적인 소비를 원하는 가성비 추구 문화 확산

지역 정보 제공 및 맞춤형 추천 서비스를 통한 디지털 지역상품권 플랫폼 개발



• System Overview



Quality Attribute: Availability, Deployability, Integrability, Energy Efficiency, Modifiability, Performance, Safety, Security, Testability, Usability

[Ex3.1] Raw QAS (Raw Quality Attribute Scenario) 발굴

- **Scenario Brainstorming:** 팀당 10개 내외(1인당 2건 ↑ 필수)의 Quality Attribute Raw 시나리오를 작성하세요.
- **Voting:** 팀당 4개 이상의 Scenario를 선정. 단, 다양한 품질 속성(3개 이상)이 나오도록 할 것

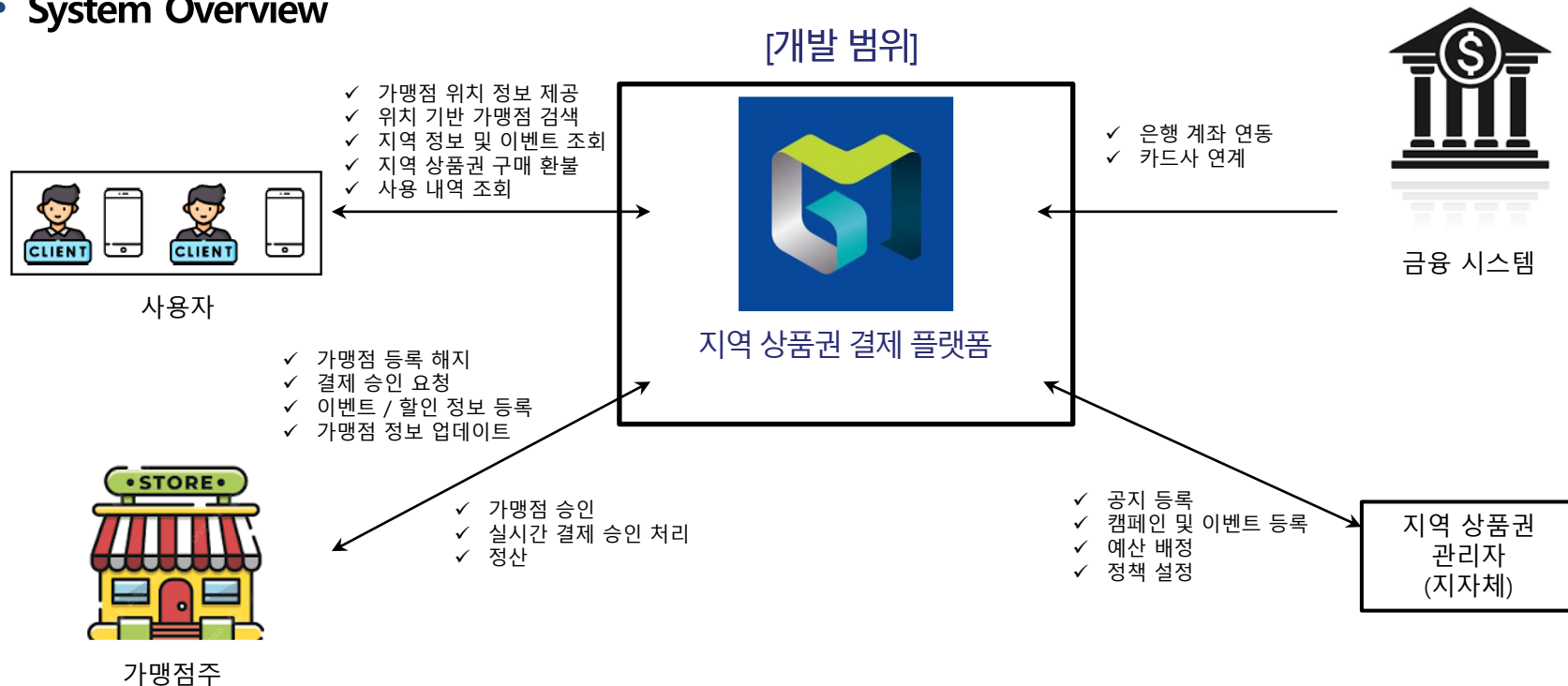
[Ex3.2] QAS (Quality Attribute Scenario) 작성

- **Scenario Refinement:** 선택된 시나리오에 대해서 품질 속성 시나리오(QAS)를 작성하세요
(문서 QA별 Guide를 참고하여 작성하세요) [12 \(참고\) QAS Guide](#)

[Ex3.3] Utility Tree 작성

- **Prioritizing:** 작성된 시나리오에 대해서 Importance/Difficulty를 부여하고 우선순위를 도출하세요. (Voting 등)

• System Overview



- **System Information**

- **Actor**

- 사용자, 가맹점주, 지역 상품권 관리자

- **Passive Actor**

- 금융 시스템

- **Non Functional Requirements (NFR)**

- **Quality Attributes**

- 20만명 이상의 사용자가 동시에 상품권 구매 시 무중단 서비스 지원
 - 20만명의 사용자가 동시에 상품권 구매 요청 시, 평균 대기시간 5 분 이내
 - 지자체별 정책 변경 비용 최소화
 - 인접 지자체를 통합 운영하게 될 경우 자원의 규모 확장이 유용한 정도

- Functional Requirement (FR)

- 사용자 (Actor 1)

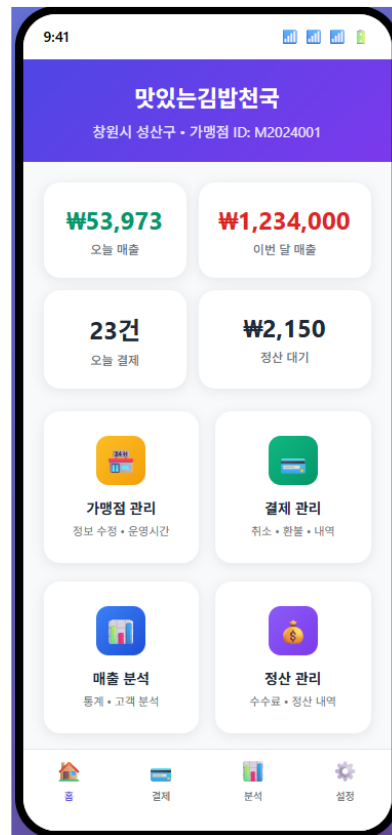
- 회원 가입 및 로그인
- 상품권 구매, 결제, 환불
 - . 계좌/카드 연동 충전
 - . 정기 충전 설정
 - . 상품권 모바일 결제
 - . 사용 내역 조회
 - . 30일 내 환불
- 가맹점 검색
 - . 위치 기반 가맹점 검색
 - . 실시간 영업 정보 확인
 - . 가맹점 위치 정보 제공
 - . 맞춤형 추천



- Functional Requirement (FR)

- 가맹점주 (Actor 2)

- 가맹점 등록 및 관리
 - . 온라인 가맹점 신청 / 해지
 - . 가맹점 정보 수정
- 정산 관리
 - . 실시간 매출 확인
 - . 수수료 내역 확인
- 결제 관리
 - . 결제 취소 및 환불 처리
 - . 일일 결제 내역 조회

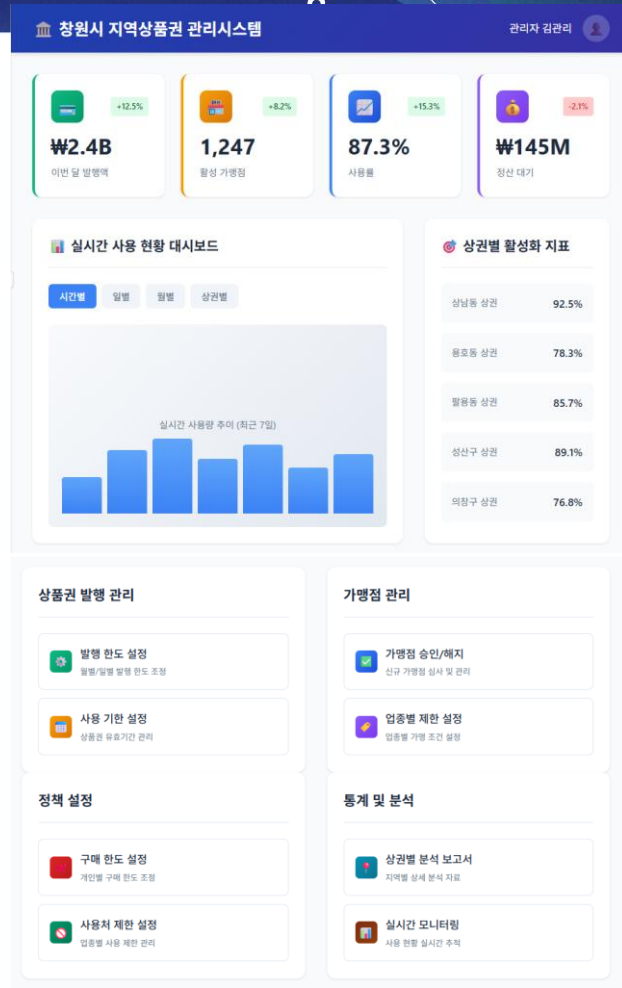


- **Functional Requirement (FR)**

- **지역 상품권 관리자 (Actor 3)**

- 상품권 발행 관리
 - . 발행 한도, 사용 기한 설정
 - . 캠페인 및 이벤트 등록
 - 가맹점 관리
 - . 가맹점 승인 및 해지
 - . 업종별 가맹 제한 설정
 - 정책 설정
 - . 구매 한도 설정
 - . 사용처 제한 설정

- Functional Requirement (FR)
 - 사용자 (Actor 1)
 - 상품권 선물
 - 가맹점주 (Actor 2)
 - 매출 분석 기능
 - . 일/주/월별 매출 통계
 - . 고객 연령대 분석
 - 지역 상품권 관리자 (Actor 3)
 - 통계 및 분석
 - . 가맹점 현황 및 분포 제공
 - . 실시간 사용 현황 대시보드 조회



- **Software for the system must :**

- 지역 상품권 구매 및
 - 사용자는 앱을 통해, **지역 상품권 구매**
 - 지역 선택 및 지역 정책에 맞게 지역 상품권 구매
 - 지역 상품권 현재 **잔액 확인 및 추가 구매/환불**
- 가맹점 검색
 - 사용자의 위치 기반 지역 상품권 **가맹점 검색**함
 - 실시간으로 영업중인 가맹점을 제공 (**5초 주기** 업데이트)
 - 특정 가맹점을 선택하면 가맹점의 위치 정보 제공
 - 사용자의 검색 키워드에 따라 위치 기반 가까운 순으로 **가맹점을 검색**
 - 사용자의 구매 이력 기반 **가맹점을 추천**
- 지역 상품권 결제
 - 사용자는 앱을 통해 지역 상품권을 이용하여 결제
 - 가맹점이 아닌 경우, 금액이 충분하지 않는 경우 결제가 이루어지지 않음
 - 가맹점주는 결제 금액이 맞지 않는 등의 이유로 결제 취소 가능

[03 Architecture설계] Ex4. Architecture View 실습 (Top Level Design) ⁺。

- Top Level Design Architecture를 UML 모델을 이용하여 작성하세요

- Top Level Design의 Structural View를 Component (or Composite Structure)Diagram 으로 작성
- Top Level Design의 Element간 동적 상호작용을 Sequence Diagram으로 명세

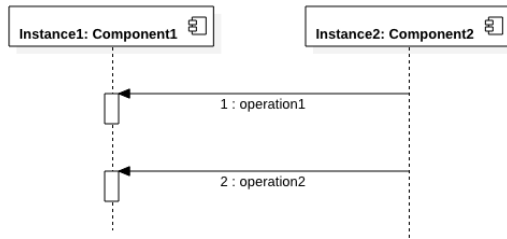


- **[필수 작성]**

- 1) 사용자가 **지역**을 선택하여 지역 정책을 확인 후 **지역 상품권 잔액을 확인**하고 추가 구매하는 동작
- 2) 사용자가 현재 **위치 기반 영업중인 가맹점**을 확인하고,
 검색 키워드를 입력하여 가맹점 리스트를 확인하고,
 특정 가맹점을 선택하여 가맹점 위치 정보를 얻는 동작

- **[선택]**

- Top Level Design의 Component 혹은
 Connector Type (인터페이스 포함)을 구체적으로 작성

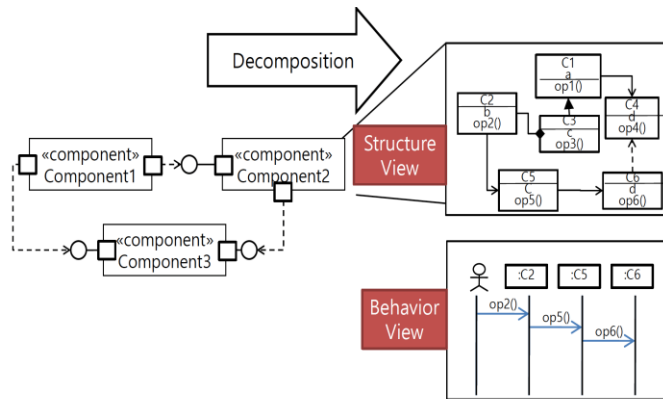


[03 Architecture설계] Ex5. Architecture View 실습 (Component Level Design)

- Top Level Design Architecture의 특정 Element에 대한

Component Level Design Architecture를 UML 모델을 이용하여 작성하세요

- Top Level Design Architecture 중 핵심 Element에 대해
Component Level Design의 Structural View를 [Class Diagram](#) 으로 작성
- Component Level Design의 Behavior View를 (동적 상호작용) [Sequence Diagram](#)으로 명세 (2개 이상)
- Top Level Architecture와 Component Level Architecture의 Structural View, Behavior View의 일관성 유지



- Ex6-7: Architecture Tactic 작성

- 실습 교안 참고하여, 실습 Template 작성

- Ex8: Architecture Design 작성

- 실습 교안 참고하여, 실습 Template 작성

- OOAD과목에서 설계한 Project에 대해 설계패턴 5가지를 적용하시오.
 - (a) 기존 설계안에서 2개 적용
 - (b) 추가 Requirement를 고안하고 이에 대한 확장을 위해 필요한 패턴을 3개 이상 적용
 - (c) Step (a)와 (b)에서 적용한 패턴의 개수가 5개 이상이어야 함
 - (d) 적용된 패턴의 설명을 위해 Class Diagram, Sequence Diagram 등을 이용 (효과적인 UML 다이어그램을 상황에 맞게 선택)
 - (e) 각 패턴을 적용한 합당한 근거를 제시
 - 특히 어떠한 변화가 예상되며, 왜 이러한 설계가 적합하다고 생각하는지 설명 필수 기입

[04 Design Patterns] Overall Class diagram

- 5개 패턴이 모두 적용된 전체 그림을 1장으로 표현
- Class Name과 Relationship 중심으로 표현 (단, dependency는 제외 가능)

